

SE3082 – Parallel Computing

Exercise 3



IT24103470
Kathisan A

Exercise 3 Details, Code & Output Verification

1. Exercise Task Description

Run the provided code and describe how the `#pragma omp master` command works in this program.

2. Complete C Source Code (exercise3.c)

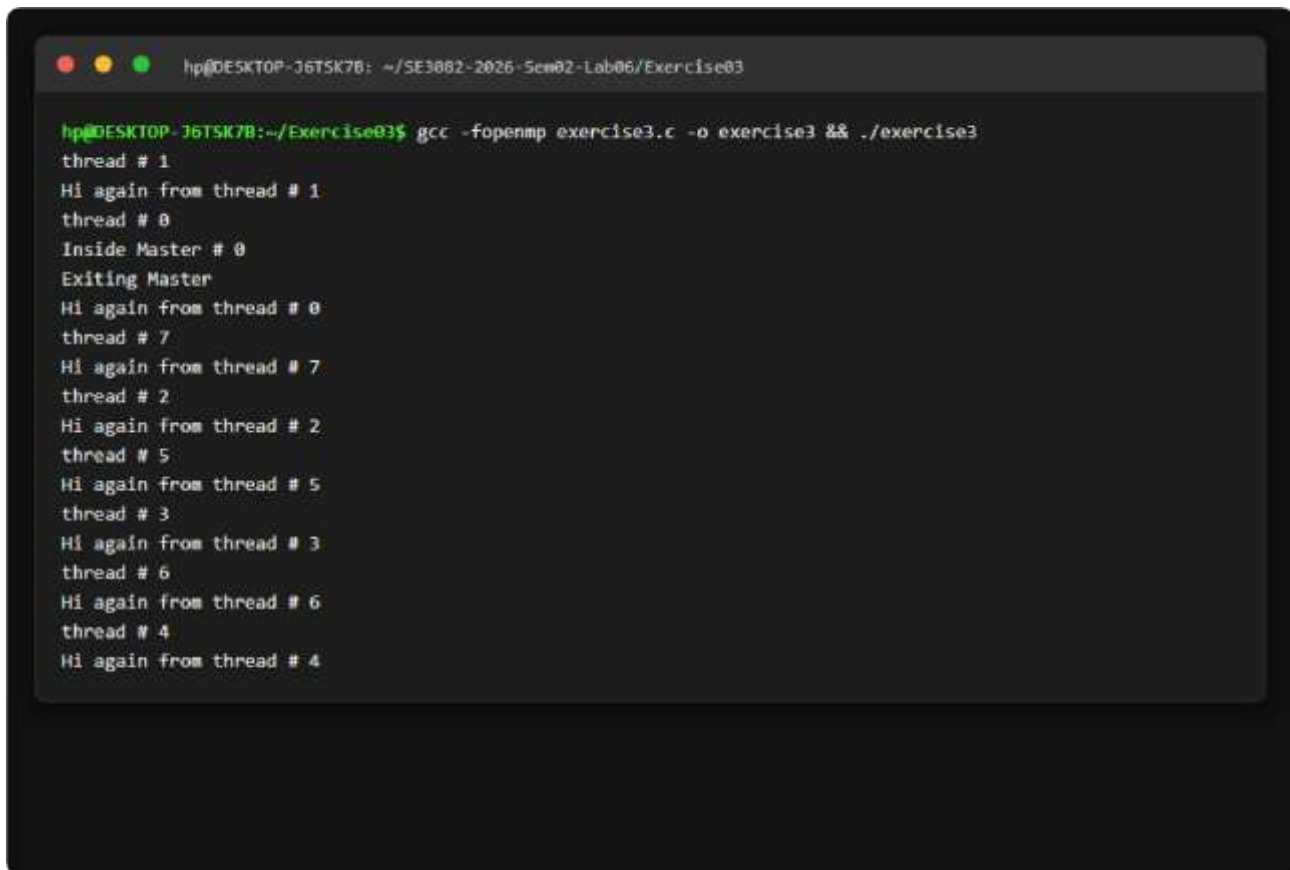
```
#include <omp.h>
#include <stdio.h>

int main() {
#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

    #pragma omp master
    {
        printf("Inside Master # %d\n", omp_get_thread_num());
        printf("Exiting Master \n");
    }

    printf("Hi again from thread # %d\n", omp_get_thread_num());
}
    return 0;
}
```

3. Terminal Execution Output Screenshot

A terminal window with a dark background and light green text. The window title is 'hp@DESKTOP-J6TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise03'. The command executed is 'gcc -fopenmp exercise3.c -o exercise3 && ./exercise3'. The output shows a sequence of messages from different threads: thread # 1, thread # 0, thread # 7, thread # 2, thread # 5, thread # 3, thread # 6, and thread # 4. The messages include 'Hi again from thread # 1', 'Inside Master # 0', 'Exiting Master', and 'Hi again from thread # 0'.

```
hp@DESKTOP-J6TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise03
hp@DESKTOP-J6TSK7B:~/Exercise03$ gcc -fopenmp exercise3.c -o exercise3 && ./exercise3
thread # 1
Hi again from thread # 1
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 7
Hi again from thread # 7
thread # 2
Hi again from thread # 2
thread # 5
Hi again from thread # 5
thread # 3
Hi again from thread # 3
thread # 6
Hi again from thread # 6
thread # 4
Hi again from thread # 4
```

4. Directive Analysis & Explanation

How #pragma omp master works:

The #pragma omp master directive specifies that the enclosed structured block is executed **only by the master thread (thread 0)**. Unlike omp single, there is **NO implicit barrier** at the end of the master block, so other threads skip it immediately and continue execution.